



VISVESVARAYA TECHNOLOGICAL UNIVERSITY, BELAGAVI

(A State University of Government of Karnataka Established as per the VTU Act, 1994)

Submission of Synopsis for

MAJOR PROJECT PHASE – 1

AI-Powered Vulnerability Detection And Mitigation System

BATCH: 7

1RN23CY035

Nithyashree R

1RN23CY014

Dev Kukreja

1RN23CY023

Kavindra Nishod

Under the guidance of

Dr. Manohar P

Dept of CSE-CY

Submitted on: _____

Department of Computer Science and Engineering (Cyber Security)



Estd : 2001

RN SHETTY TRUST®

RNS INSTITUTE OF TECHNOLOGY

Autonomous Institution affiliated to Visvesvaraya Technological University, Belagavi

Approved By AICTE, New Delhi. Accredited by NAAC 'A+' Grade,

Channasandra, Dr. Vishnuvardhan Road, Bengaluru - 560 098

Ph:(080)28611880, 28611881, URL: www.rnsit.ac.in

INTRODUCTION

In the current digital landscape, the complexity of software systems and the sophistication of cyber threats are evolving at an unprecedented pace. Traditional vulnerability management often relies on manual scanning and reactive patching, which can be slow and prone to human error. This project, AI-Powered Vulnerability Detection And Mitigation System, aims to bridge this gap by leveraging Artificial Intelligence (AI) and Machine Learning (ML) to automate the identification and remediation of security flaws. By integrating AI, the system can analyze vast amounts of code and network data to predict potential vulnerabilities before they are exploited, shifting the security paradigm from reactive to proactive defense.

Furthermore, the system incorporates advanced pattern recognition algorithms to detect anomalous behavior in real-time network traffic and application logs. By utilizing supervised and unsupervised learning models, it can identify both known vulnerabilities and zero-day threats with improved accuracy. The platform continuously learns from new threat intelligence feeds, enabling it to adapt to emerging attack vectors and evolving malware techniques.

The proposed solution also integrates automated risk scoring mechanisms to prioritize vulnerabilities based on severity, exploitability, and business impact. This ensures that critical issues are addressed promptly, minimizing potential damage and downtime. Additionally, the system provides intelligent patch recommendations and automated mitigation strategies to reduce manual intervention.

PROBLEM DEFINITION

Manual vulnerability assessment is time-consuming and often fails to keep up with the continuous delivery cycles of modern software development. Existing tools may generate high volumes of false positives, overwhelming security analysts. There is a critical need for an intelligent system that not only detects vulnerabilities with high accuracy but also suggests or automates mitigation strategies to reduce the window of exposure.

To address this challenge, the proposed AI-powered system integrates seamlessly into DevOps pipelines, enabling continuous security assessment alongside continuous integration and deployment processes. By embedding intelligent scanning mechanisms within the development lifecycle, vulnerabilities can be identified at the earliest stages of code creation. The system leverages contextual analysis to significantly reduce false positives, ensuring that security teams focus only on genuine threats.

OBJECTIVES & SCOPE

Objectives

1. **To Implement Automated Dependency Extraction:** Develop a system capable of parsing dependency manifest files (e.g., package.json, pom.xml, requirements.txt) from GitHub repositories and extracting all third-party libraries with their respective versions.
2. **To Integrate CVE Intelligence Sources:** Connect the platform with trusted vulnerability databases such as NIST NVD and Snyk APIs to identify known vulnerabilities associated with extracted dependencies.
3. **To Enable Context-Aware Risk Prioritization:** Design a semantic analysis pipeline using vector embeddings and LLM-based reasoning to prioritize vulnerabilities based on repository context instead of relying solely on CVSS base scores.
4. **To Generate AI-Driven Remediation Strategies:** Develop an intelligent remediation module that suggests safe upgrade paths, alternative package recommendations, or compensating controls when no patched version exists.

Scope

The project focuses specifically on dependency-level vulnerability analysis in modern software projects. It does not analyze application source code logic flaws but concentrates on third-party package risks.

- **GitHub Repository Integration:** Secure ingestion and analysis of public repositories.
- **Dependency Manifest Analysis:** Extraction of package names and pinned versions.
- **CVE Detection:** Identification of known vulnerabilities through authoritative databases.
- **Vector-Based Semantic Retrieval:** storage of vulnerability embeddings in ChromaDB for contextual similarity search.
- **LLM-Based Contextual Risk Assessment:** Intelligent prioritization based on exploitability and repository usage context.
- **Automated Reporting Dashboard:** Structured vulnerability reports with ranked findings and actionable mitigation guidance.

Limitations

- The system focuses only on known CVEs available in external databases and may not detect zero-day vulnerabilities.
- Analysis is limited to dependency-level security risks and does not detect application logic flaws such as business logic vulnerabilities.
- Accuracy of risk prioritization depends on the quality and freshness of external CVE intelligence feeds.

- Private repositories may require secure authentication and access control mechanisms.

LITERATURE REVIEW

1. Dependency Vulnerability in Modern Software Ecosystems

Modern software development heavily relies on third-party open-source libraries. Research by Decan et al. (ICSME 2018) highlights the concept of technical lag, where projects continue using outdated dependencies despite available updates. Studies show that transitive dependencies significantly increase the attack surface, making software supply chains vulnerable to exploitation. This emphasizes the need for automated dependency-level vulnerability analysis rather than focusing solely on application code.

2. Limitations of Traditional CVE Scanners

Conventional vulnerability scanners such as OWASP Dependency-Check and Snyk primarily rely on matching package names and versions against known CVE databases. While effective in detection, research by Pashchenko et al. (ICSE 2018) demonstrates that many reported vulnerabilities are not actually exploitable in a specific project context. This results in excessive alerts and inefficient prioritization, highlighting the need for context-aware vulnerability assessment rather than severity-score-based filtering alone.

3. Risk Prioritization Beyond CVSS Scores

The Common Vulnerability Scoring System (CVSS) provides standardized severity ratings, but several studies argue that CVSS base scores do not reflect real-world exploitability. Sabottke et al. (USENIX Security Symposium) showed that exploit prediction requires contextual intelligence, including threat activity and code usage patterns. This supports the adoption of intelligent prioritization mechanisms that consider repository-specific context and exploit likelihood rather than relying purely on numeric severity scores.

4. AI and Semantic Retrieval in Security Analysis

Recent advancements in Natural Language Processing (NLP) and vector databases have enabled semantic similarity search for security intelligence. Embedding-based retrieval systems, such as those using vector stores, allow efficient matching of contextual vulnerability descriptions. Research indicates that integrating Large Language Models (LLMs) with structured vulnerability data enhances reasoning capabilities and enables automated remediation suggestions. This emerging approach extends beyond traditional static scanners by combining retrieval mechanisms with contextual AI reasoning.

METHODOLOGY

System Architecture

The project follows a pipeline-based, AI-driven architecture integrating repository ingestion, vulnerability intelligence retrieval, semantic vector storage, and LLM-based contextual reasoning. The system is structured into four major layers:

1. Repository Ingestion Layer
2. Vulnerability Intelligence Layer
3. AI Processing & Vector Analysis Layer
4. Presentation & Reporting Layer

The architecture is deployed as a full-stack application with a React frontend, Spring Boot backend, PostgreSQL database, ChromaDB vector store, and external LLM API integration. The workflow is modular, scalable, and suitable for DevSecOps environments.

Technical Approach

1. **Repository Ingestion (Data Collection Phase)**
 - Users connect a public GitHub repository to the platform.
 - The backend parses dependency manifest files such as package.json, pom.xml, and requirements.txt.
 - All third-party dependencies along with their pinned versions are extracted.
 - Extracted data is stored in PostgreSQL for structured tracking and further analysis.
2. **Vulnerability Detection (CVE Intelligence Integration):** Each extracted dependency and its specific version are systematically compared against trusted vulnerability databases such as NVD and Snyk through secure API integration. Relevant vulnerability details, including CVE identifiers, CVSS scores, exploit metadata, and descriptive information, are retrieved for matching entries. The system establishes an accurate mapping between dependency-version pairs and their associated vulnerability records. All collected data is stored in a structured format to enable efficient downstream AI-based analysis and risk prioritization.

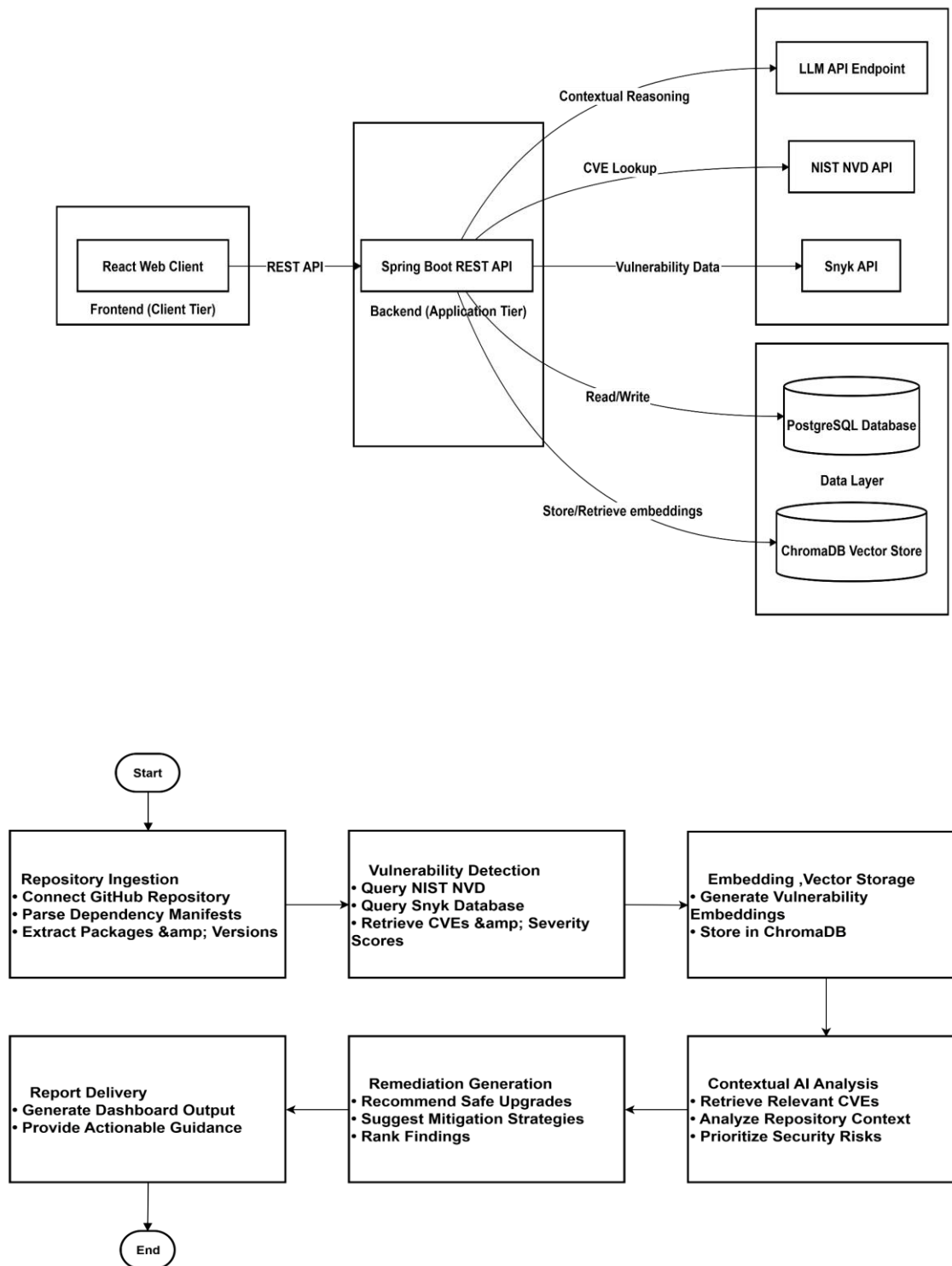


Figure 1. System Architecture and Data flow diagram

3. **Embedding, Vector Storage and Contextual AI Analysis:** This Retrieved vulnerability descriptions and related metadata are converted into vector embeddings using advanced embedding models. These embeddings are stored in ChromaDB to enable semantic similarity-based retrieval instead of simple keyword matching. The Large Language Model retrieves contextually relevant embeddings and combines them with repository-specific signals such as dependency usage and exposure level. Based on this enriched context, the system generates a prioritized risk assessment

that considers exploitability and real-world impact beyond static CVSS scores.

4. **Remediation Generation (Automated Mitigation Strategy):** For each prioritized vulnerability, the system recommends a secure upgrade path whenever a patched version is available. In cases where no safe version exists, alternative mitigation strategies such as dependency replacement or compensating security controls are suggested. The AI model analyzes contextual information to generate practical and repository-specific recommendations. All remediation guidance is structured clearly to ensure it is actionable for developers.to demonstrate detection capabilities.

RESOURCES/REQUIREMENTS

Hardware Requirements

- Processor: Intel Core i5/i7 or equivalent (recommended for handling vector embeddings and AI processing workloads).
- RAM: Minimum 8 GB (16 GB recommended for smooth execution of backend services, database, and vector storage).
- Storage: Minimum 20–30 GB free space for application deployment, database storage, and vulnerability datasets.
- Internet Connectivity: Stable broadband connection required for GitHub repository access and external API calls (NVD, Snyk, LLM services).

Software Requirements

- Operating System: Windows 10/11, Ubuntu Linux, or macOS.
- Backend Framework: Spring Boot (REST API development and business logic implementation).
- Frontend Framework: React.js (Dashboard and user interface development).
- Programming Languages: Java (Backend), JavaScript (Frontend).
- Database: PostgreSQL (Primary relational database for structured data storage).
- Vector Database: ChromaDB (Semantic embedding storage and similarity retrieval).
- LLM Integration: External LLM API (for contextual reasoning and remediation generation).
- APIs & External Services: NIST NVD API and Snyk API for vulnerability intelligence retrieval.

- **Version Control & Repository Access:** GitHub integration for dependency extraction.

EXPECTED OUTCOMES

The primary deliverable of the project is a fully functional AI-driven dependency vulnerability analysis platform capable of scanning GitHub repositories and generating context-aware security reports. Specific expected outcomes include:

1. **Accurate Dependency Vulnerability Detection:** The system will successfully identify known CVEs associated with third-party dependencies by cross-referencing extracted package versions with trusted vulnerability databases such as NVD and Snyk.
2. **Context-Aware Risk Prioritization:** Instead of presenting vulnerabilities purely based on CVSS scores, the system will rank findings according to repository-specific context, exploit likelihood, and practical impact, thereby reducing irrelevant alerts.
3. **AI-Generated Remediation Guidance:** The platform will provide actionable mitigation strategies, including secure upgrade paths, alternative package recommendations, and compensating controls where no safe version exists.
4. **Interactive Reporting Dashboard:** A web-based dashboard will display structured vulnerability reports, prioritized risk levels, CVE details, and remediation recommendations, along with scan history for comparative analysis.
5. **End-to-End Demonstration Capability:** The system will demonstrate successful scanning of intentionally vulnerable repositories (e.g., OWASP-based test projects), clearly showing detection accuracy, intelligent prioritization, and automated remediation generation in real time.

APPLICATIONS

1. **Software Development Teams:** Development teams can use the platform to identify outdated or insecure third-party packages and receive AI-generated remediation guidance, reducing security debt and improving codebase reliability.
2. **Security Operations Centers (SOC):** Security analysts can leverage context-aware vulnerability prioritization to focus on high-risk, exploitable issues instead of responding to large volumes of low-impact CVE alerts.
3. **Academic and Research Institutions:** The system can serve as a practical research tool for studying software supply chain security, vulnerability prioritization techniques, and AI-driven remediation strategies.

References

- [1] M. S. G. K. K. P. R., "Security Information and Event Management (SIEM): Analysis, Trends, and Usage," IEEE Access, vol. 8, 2020.
- [2] R. Chandramouli, "Application Container Security Guide," NIST SP 800-190, 2019.
- [3] K. Scarfone and P. Mell, "Guide to Intrusion Detection and Prevention Systems (IDPS)," NIST SP 800-94, 2007.
- [4] S. Salah, G. Maci^o-Fern^ondez, and R. A. Rodriguez-Gomez, "A Model-based Survey of Alert Correlation Techniques," Computer Networks, vol. 57, no. 4, 2013.
- [5] MITRE Corporation, "MITRE ATT&CK Framework: Container Matrix," 2023. [Online]. Available: <https://attack.mitre.org>
- [6] A. Al-Shourbaji et al., "Integrated Multilayered Framework for IoT Security Intrusion," Intelligent Automation & Soft Computing, vol. 36, no. 1, 2023.
- [7] S. M. Kasongo, "Ensemble Defense System: A Hybrid IDS Approach," arXiv:2401.03491, 2024.
- [8] "Autonomous Cyber-Physical Security Middleware for IoT," Frontiers in Artificial Intelligence, 2025.

